



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

JACKFRUIT MINI-PROJECT — UE24CS341A

Software Engineering

Software Requirements Specification

For

V.A.R.I.O

(Voice Adaptive Role & Intent Orchestrator)

A Role-Specific Voice Assistant Framework for Organizations and Entities

Project ID: 42

Team name : PIPELINE

Team Details:

NAME	SRN
Rohith G S	PES2UG24AM138
Kurumeti Mitesh	PES2UG24AM080
Mohammed Tahir Siddique	PES2UG24AM092
Pavan Kishor M	PES2UG24AM111

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
1.1 Problem Statement	4
1.2 Objectives	4
1.3 Scope	5
1.4 Stakeholders / Users	5
2. Functional Requirements.....	6
2.1 Core Intent Recognition & Role-Routing Engine	6
2.2 HR Role Pack	6
2.3 IT Support Role Pack	6
2.4 Admissions Role Pack	6
2.5 Admin Configuration	7
3. Non-Functional Requirements.....	8
3.1 Performance	8
3.2 Safety	8
3.3 Security	8
3.4 Reliability & Availability	8
3.5 Usability	8
3.6 Maintainability & Extensibility	8
3.7 Testability	8
4. System Features.....	9
4.1 Multi-Turn Conversational Intent Recognition	9
4.2 HR Self-Service (Leave & Policy)	9
4.3 IT Support Self-Service (Tickets & Password Reset)	9
4.4 Admissions Self-Service (Status & Requirements)	9
4.5 Role-Pack Configuration Console	9
4.6 Voice Interaction	9
5. Use Cases.....	10
UC-1: Check Leave Balance	10
UC-2: Submit a Leave Request	10
UC-3: Raise an IT Support Ticket	10
UC-4: Check IT Ticket Status	11
UC-5: Check Admissions Application Status	11
UC-6: Configure a Role Pack's Intents/FAQs	11
6. System Architecture.....	12
6.1 Layer Descriptions	12
7. Technology Requirements.....	13

8. Constraints / Assumptions.....	14
8.1 Constraints	14
8.2 Assumptions	14
9. Future Enhancements.....	15

1. Introduction

V.A.R.I.O. (Voice Adaptive Role & Intent Orchestrator) is a role-specific, context-aware voice and text assistant framework designed for organizations and institutions. Rather than being a single-purpose chatbot, V.A.R.I.O. is built as a customizable core engine with pluggable "role packs" — allowing the same underlying system to be configured for different organizational functions such as Human Resources (HR), IT Support, and Admissions, each with its own vocabulary, knowledge base, and workflows.

1.1 Problem Statement

Organizations such as universities and companies field a large volume of repetitive, low-complexity queries and simple transactional requests directed at HR, IT Support, and Admissions teams — e.g., "What is my leave balance?", "How do I reset my password?", "What is the status of my application?". These queries consume staff time that could be spent on higher-value work, and employees/students/applicants often face delays waiting for a human response to questions that have a well-defined, repeatable answer or workflow. Existing chatbot solutions are typically built as one-off, single-domain bots, meaning organizations must build and maintain a separate system for each department, duplicating engineering effort and making it hard to introduce a new departmental assistant quickly. There is a need for a single, reusable assistant framework whose conversational "brain" is shared across roles, while each role's specific knowledge and workflows remain independently configurable.

1.2 Objectives

- To design and implement a role-agnostic core engine capable of natural-language intent recognition and context-aware conversation routing.
- To build three initial, fully functional role packs — HR, IT Support, and Admissions — each handling representative FAQ and workflow-style queries for that domain.
- To provide a swappable integration-adaptor layer so that internal-tool integrations (mocked for this project) can later be replaced with real enterprise systems (HRMS, ITSM, Admissions software) without altering the core engine.
- To support both text and voice interaction, making the assistant usable in a chat-widget style interface as well as hands-free.
- To demonstrate that a new role or workflow can be added primarily through configuration rather than by rewriting core logic, proving the framework's extensibility.
- To apply complete SDLC practice — Agile sprints, Git-based version control, CI/CD, code review, and structured testing — throughout the build.

1.3 Scope

- A core NLU/intent-recognition and routing engine shared by all role packs.
- Three role packs: HR (leave balance/status, policy FAQs, leave-request submission), IT Support (ticket creation, ticket-status lookup, password-reset flow), and Admissions (application-status lookup, document checklist, fee/deadline FAQs).
- Mocked internal-tool APIs standing in for a real HRMS, ITSM, and Admissions system, built behind a common adapter interface.
- A web-based chat interface (React) supporting both typed and spoken input, and a basic admin configuration panel for editing intents/FAQs per role.

Out of scope for Version 1.0 (see Section 9, Future Enhancements):

- Integration with a real, live enterprise HRMS/ITSM/Admissions system.
- Support for languages other than English.
- A native mobile application (v1 is web-only, mobile-responsive).
- Advanced analytics/BI dashboards beyond basic conversation and workflow logs.

1.4 Stakeholders / Users

Stakeholder / User Class	Role in the System
End User — Employee	Uses V.A.R.I.O. to check HR information (leave balance, policies) and IT support (tickets, password resets) relevant to their employment.
End User — Student / Applicant	Uses V.A.R.I.O. to check admissions application status, document requirements, and fee/deadline information.
Role Administrator (HR / IT / Admissions Admin)	Configures and maintains the intents, FAQ knowledge base, and workflow rules for their specific role pack.
System Administrator (Development Team)	Deploys, monitors, and extends the V.A.R.I.O. framework; owns the Core Engine and Integration Adapter layer.
Course Instructor / Evaluator	Reviews requirements, design, and implementation artefacts, and evaluates the final working system and demo.

2. Functional Requirements

Functional requirements are grouped by the module they belong to.

2.1 Core Intent Recognition & Role-Routing Engine

- **FR-1:** The system shall classify an incoming user utterance into one of the predefined intents for the active role, returning an intent label and a confidence score.
- **FR-2:** If the top intent's confidence score falls below a configurable threshold, the system shall respond with a clarification prompt instead of executing a workflow.
- **FR-3:** The system shall support registering a new role pack (its intents, FAQs, and workflow bindings) via configuration data, without requiring a code change or redeployment of the core engine.
- **FR-4:** The system shall persist conversation context (role, prior intents, partially filled workflow slots) for the duration of a session and correctly resume a multi-turn workflow across turns.
- **FR-5:** The system shall log every classified intent, confidence score, and routing decision for audit and demo traceability.

2.2 HR Role Pack

- **FR-6:** The system shall retrieve an employee's leave balance and leave-application status from the mock HRMS API given a valid employee identifier.
- **FR-7:** The system shall answer HR policy FAQ questions from a curated, configurable knowledge base.
- **FR-8:** The system shall allow a user to submit a leave request through a guided conversational flow (leave type, start date, end date, reason) with a confirmation step before submission.
- **FR-9:** On successful leave-request submission, the system shall return a unique request reference ID.

2.3 IT Support Role Pack

- **FR-10:** The system shall create an IT support ticket via the mock ITSM API, capturing category, description, and reporter identifier, and return a unique ticket ID.
- **FR-11:** The system shall retrieve and display the current status of an existing ticket given a valid ticket ID.
- **FR-12:** The system shall guide a user through a password-reset request flow, including a mocked identity-verification step, before confirming the reset has been triggered.
- **FR-13:** The system shall prevent duplicate ticket creation for an identical, immediately repeated request within the same session.

2.4 Admissions Role Pack

- **FR-14:** The system shall retrieve and display an applicant's current application status given a valid application ID.
- **FR-15:** The system shall provide a document checklist appropriate to the program the applicant selects.
- **FR-16:** The system shall answer fee- and deadline-related questions from a single, configurable admissions-calendar data source.
- **FR-17:** The system shall clearly indicate when requested application data cannot be found and suggest a next step rather than failing silently.

2.5 Admin Configuration

- **FR-18:** The system shall allow an authenticated Role Administrator to add, edit, or remove intents and FAQ entries for their own role pack only.
- **FR-19:** The system shall allow a Role Administrator to view a log of conversations and workflow actions triggered within their role.

3. Non-Functional Requirements

3.1 Performance

- For 95% of queries (excluding artificially delayed mock API calls), the system shall return a response within 2 seconds of the user finishing their utterance.
- The system shall support at least 20 concurrent user sessions during the demo without noticeable response-time degradation.

3.2 Safety

- No workflow that would materially affect a real record (leave approval, ticket closure) shall execute without an explicit user confirmation step.
- Every workflow-triggering action shall be written to an audit log with timestamp, user identifier, and action taken.

3.3 Security

- Access to the admin configuration panel shall require JWT-based authentication; unauthenticated requests shall be rejected.
- Role administrators shall only view/edit configuration for their own role pack (role-based access control).
- All user input shall be validated/sanitised before being passed to the NLU pipeline or persisted, to prevent injection attacks.
- No personally identifiable information shall be persisted beyond the active session unless the workflow explicitly requires it.

3.4 Reliability & Availability

- The system shall degrade gracefully if a mock internal-tool API is unavailable, informing the user rather than crashing the conversation.
- The Core Engine shall recover an in-progress session's context after a transient backend restart, where session data has been persisted.

3.5 Usability

- The assistant shall support both text and voice input, with a text fallback whenever the browser lacks voice support.
- Error and fallback messages shall be clear and jargon-free, guiding the user toward a next step.

3.6 Maintainability & Extensibility

- The Core Engine, each Role Pack, and the Integration Adapter layer shall be implemented as separate, independently testable modules/services.
- Adding a new role pack shall require configuration changes only, achievable in under one developer-day, without core-engine code changes.

3.7 Testability

- Every functional requirement in Section 2 shall have at least one corresponding automated or manual test case.

4. System Features

4.1 Multi-Turn Conversational Intent Recognition

Priority: *High*

Accepts free-text or transcribed-voice input, classifies role and intent with a confidence score, and maintains context across multiple conversation turns so that workflows requiring several pieces of information (e.g., a leave request) can be completed incrementally.

4.2 HR Self-Service (Leave & Policy)

Priority: *High*

Lets an employee check leave balance/status, ask HR policy questions, and submit a leave request conversationally, ending with a reference ID for tracking.

4.3 IT Support Self-Service (Tickets & Password Reset)

Priority: *High*

Lets an employee raise an IT ticket, check an existing ticket's status, or go through a guided password-reset flow, without needing to open a separate ITSM portal.

4.4 Admissions Self-Service (Status & Requirements)

Priority: *High*

Lets an applicant check application status, retrieve a document checklist for their chosen program, and get fee/deadline information sourced from a single configurable calendar.

4.5 Role-Pack Configuration Console

Priority: *Medium*

Gives a Role Administrator a simple interface to add/edit intents and FAQ entries for their domain, and to review recent conversation/workflow logs, without needing developer involvement.

4.6 Voice Interaction

Priority: *Medium*

Allows a user to speak their query using the browser's Web Speech API; the transcribed text is passed through the same NLU pipeline as typed input, and (optionally) the response can be read back aloud.

5. Use Cases

UC-1: Check Leave Balance

Actor(s): *End User (Employee)*

Precondition: *User has selected the HR role and provided a valid employee identifier during the session.*

Main Flow:

- 1. User asks, e.g., "What is my leave balance?"
- 2. Core Engine classifies the intent as `check_leave_balance` with high confidence.
- 3. HR Role Pack calls the mock HRMS adapter with the employee identifier.
- 4. Mock HRMS API returns the current leave balance.
- 5. System formats and returns the balance to the user in natural language.

Postcondition: *User receives an accurate, current leave balance; the interaction is logged.*

UC-2: Submit a Leave Request

Actor(s): *End User (Employee)*

Precondition: *User has selected the HR role and is authenticated for the session.*

Main Flow:

- 1. User states intent to apply for leave.
- 2. System asks for leave type, start date, end date, and reason in turn (multi-turn slot filling).
- 3. System presents a summary of the collected details for confirmation.
- 4. User confirms; system submits the request via the mock HRMS adapter.
- 5. System returns a unique request reference ID to the user.

Postcondition: *A leave request is recorded (in the mock HRMS) with a reference ID; the action is written to the audit log.*

UC-3: Raise an IT Support Ticket

Actor(s): *End User (Employee)*

Precondition: *User has selected the IT Support role.*

Main Flow:

- 1. User describes an issue, e.g., "My laptop won't connect to WiFi."
- 2. Core Engine classifies the intent as `raise_ticket`.
- 3. IT Role Pack asks for/derives category and a short description.
- 4. System calls the mock ITSM adapter to create the ticket.
- 5. System returns the generated ticket ID to the user.

Postcondition: *A new ticket exists in the mock ITSM system with a unique ID; duplicate immediate repeats are prevented.*

UC-4: Check IT Ticket Status

Actor(s): *End User (Employee)*

Precondition: *User has selected the IT Support role and has a previously issued ticket ID.*

Main Flow:

- 1. User asks for the status of a given ticket ID.
- 2. Core Engine classifies the intent as `check_ticket_status` and extracts the ticket ID entity.
- 3. IT Role Pack queries the mock ITSM adapter for the ticket's current status.
- 4. System returns the status (Open / In Progress / Resolved / Closed) to the user.

Postcondition: *User is informed of the current ticket status; if the ticket ID is invalid, the user is clearly told and guided to a next step.*

UC-5: Check Admissions Application Status

Actor(s): *End User (Applicant)*

Precondition: *User has selected the Admissions role and has a valid application ID.*

Main Flow:

- 1. User asks for their application status, providing an application ID.
- 2. Core Engine classifies the intent as `check_application_status`.
- 3. Admissions Role Pack queries the mock Admissions adapter.
- 4. System returns the current status and, where relevant, next steps.

Postcondition: *User receives accurate application status; unresolved/invalid IDs are handled gracefully with guidance.*

UC-6: Configure a Role Pack's Intents/FAQs

Actor(s): *Role Administrator*

Precondition: *Administrator is authenticated with a valid role-scoped JWT token.*

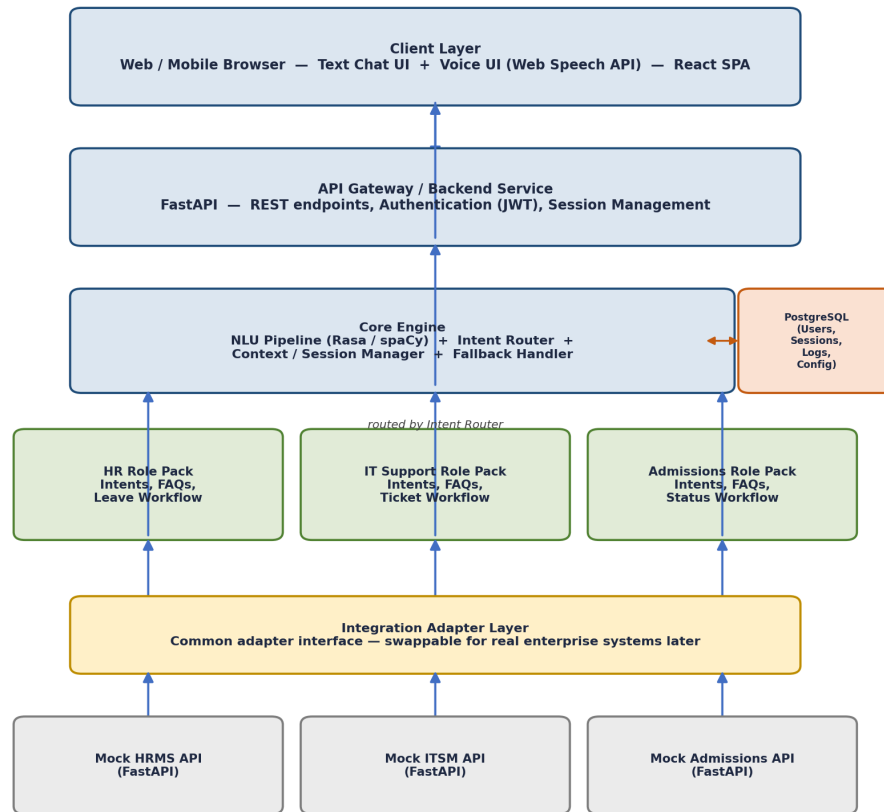
Main Flow:

- 1. Administrator logs into the configuration console.
- 2. System restricts the view to only the administrator's own role pack.
- 3. Administrator adds, edits, or removes an intent or FAQ entry.
- 4. System validates and persists the change to the role pack's configuration store.
- 5. Change is reflected in the live conversation flow without a redeployment.

Postcondition: *The role pack's configuration is updated; the change is timestamped and attributed to the administrator.*

6. System Architecture

V.A.R.I.O. follows a layered architecture that keeps the role-agnostic Core Engine separate from role-specific logic and from any internal-tool integration, so that new roles or real backend systems can be introduced with minimal disruption.



6.1 Layer Descriptions

- **Client Layer:** A React single-page application providing a text chat widget and optional voice input/output via the browser's Web Speech API.
- **API Gateway / Backend Service:** A FastAPI application exposing REST endpoints, handling authentication (JWT) and session management for each conversation.
- **Core Engine:** Role-agnostic NLU pipeline (Rasa or spaCy-based classifier), intent router, context/session manager, and fallback handler.
- **Role Packs:** Three independent modules — HR, IT Support, Admissions — each owning its own intents, FAQ content, and workflow logic.
- **Integration Adapter Layer:** A common interface abstracting calls to internal tools, so mock APIs used in this project can later be swapped for real HRMS/ITSM/Admissions systems without changing Role Pack or Core Engine code.
- **Mock Internal Systems:** FastAPI services simulating an HRMS, ITSM, and Admissions system, standing in for real enterprise software the team does not have access to.
- **PostgreSQL Database:** Stores users, session data, role-pack configuration, conversation logs, and workflow records, accessed by the Core Engine and Role Packs via an ORM (SQLAlchemy).

7. Technology Requirements

Layer / Concern	Technology	Notes
Frontend	React (JavaScript)	Chat widget UI, voice input/output via Web Speech API, admin configuration console.
Backend / API Gateway	Python 3.11+, FastAPI	REST API, authentication, session management.
NLU / Intent Recognition	Rasa Open Source (primary) or spaCy-based classifier (lightweight fallback)	Trained on a project-curated intent dataset per role.
Database	PostgreSQL 15+	Users, sessions, role-pack configuration, logs, workflow records.
Internal-tool Mocks	FastAPI (separate services)	Simulated HRMS, ITSM, and Admissions APIs behind a common adapter contract.
Version Control	Git + GitHub	Feature-branch workflow with mandatory pull-request review.
Project / Agile Tracking	GitHub Projects or Jira	User stories, sprint backlog, burndown tracking.
CI/CD	GitHub Actions / Jenkins	Automated build, test, and static-analysis pipeline on every PR.
Containerization	Docker	Consistent local/dev/demo environments for backend, NLU service, and mock APIs.
Testing	pytest (backend/unit), Jest/React Testing Library (frontend)	Unit, integration, and system test coverage.
Static Analysis	SonarQube (or equivalent)	Code quality and security scanning before merge.
Authentication	JWT (JSON Web Tokens)	Used for admin configuration console access control.

8. Constraints / Assumptions

8.1 Constraints

- The team must use Python (FastAPI) for the backend and React for the frontend, per the chosen technology direction.
- Git/GitHub with a Jira-or-similar Agile board is mandatory for tracking user stories and backlog, per course requirement.
- A CI/CD pipeline, mandatory code review (PRs), and static/security analysis must be in place before code is merged.
- The team has no access to a real enterprise HRMS, ITSM, or Admissions system; all such integrations must be mocked behind a documented adapter interface.
- The project must be completed by a team of 4 within a single university semester, alongside coursework in other subjects.
- English-language interaction only for Version 1.0.

8.2 Assumptions

- Pre-trained NLU models (Rasa or spaCy), fine-tuned on a modest hand-labelled dataset (roughly a few hundred examples per intent), will provide acceptable classification accuracy within the project's scope and timeline.
- Mock internal-tool APIs are an acceptable and sufficient substitute for real systems for the purposes of this academic project.
- End users have access to a modern desktop or mobile browser; those without microphone access or a browser lacking the Web Speech API will use text input as a fallback.
- Third-party open-source dependencies (Rasa/spaCy, FastAPI, PostgreSQL, React) remain available and version-compatible for the duration of the project.
- Team members have, or will acquire during the course, sufficient familiarity with the chosen stack to implement their owned module independently.

9. Future Enhancements

- Replace mock HRMS, ITSM, and Admissions APIs with real enterprise-system integrations, using the existing Integration Adapter interface as the connection point.
- Add multi-language support, extending the NLU pipeline and FAQ/knowledge-base content beyond English.
- Introduce additional role packs (e.g., Finance, Facilities, Library) to demonstrate further the framework's extensibility beyond the initial three roles.
- Build an analytics dashboard summarising query volume, top intents, fallback rate, and workflow completion rate per role, to help administrators identify FAQ/workflow gaps.
- Develop a native mobile application (iOS/Android) in addition to the current web-based, mobile-responsive interface.
- Add proactive/outbound notifications (e.g., notify a user when their raised ticket or leave request changes status), rather than only responding to user-initiated queries.
- Incorporate a feedback loop where low-confidence or misclassified intents are surfaced to Role Administrators to improve the underlying NLU training data over time.
- Explore single-sign-on (SSO) integration so end users are automatically authenticated using their existing organizational credentials.